Indian Institute of Technology Bhilai

Department of Electrical Engineering

Course: Digital Image Processing EEL605

DIP Project Report

Team 13

Block-Level Double JPEG Compression Detection for Image Forgery Localization

Team Members

Name	Roll Number
Anurag Jaiswal	12340300
Shivam Singh	12342020
Divit Sharma	12340680
Ramavath Srilatha	12341720

Project Instructor: Dr. Nitin Khanna

May 4, 2026

Contents

1 Introduction **2**
- 1.1 Background and Motivation . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 2

2 Mathematical Concepts **2**
- 2.1 JPEG Compression Pipeline . . . 2
  - 2.1.1 Color Space Conversion . . . 2
  - 2.1.2 Discrete Cosine Transform (DCT) . . . 3
  - 2.1.3 Quantisation . . . 3
  - 2.1.4 Entropy Encoding and JPEG Bit-stream . . . 3
  - 2.1.5 JPEG Decompression . . . 3
- 2.2 Double Quantisation Artifacts . . . 4
- 2.3 Proposed Input Representation . . . 4
  - 2.3.1 Histogram of Quantised DCT Coefficients . . . 4
  - 2.3.2 Channel-wise Concatenation with Q-factors . . . 4
- 2.4 Improvement over other methods . . . 5

3 Architecture Explanation **5**
- 3.1 Overview . . . 5
- 3.2 DenseNet-121 . . . 5
  - 3.2.1 Architecture Details . . . 5
- 3.3 Forgery Localisation Model . . . 6
- 3.4 Training Configuration . . . 6

4 Algorithm **7**

5 Results **8**
- 5.1 Effect of Training Data Size . . . 8
- 5.2 Effect of Histogram Bin Range . . . 9
- 5.3 Effect of CNN Architecture . . . 9
- 5.4 Performance at Different Patch Sizes . . . 9
- 5.5 State-of-the-Art Comparison (30% Training Split) . . . 9
- 5.6 Unseen Quantisation Matrices . . . 10
- 5.7 Quantitative Forgery Localisation . . . 10
- 5.8 Patch Localization . . . 10

6 Comparison with Paper **11**
- 6.1 Training Data Size Comparison . . . 11
- 6.2 Histogram Bin Range Comparison . . . 11
- 6.3 CNN Architecture Comparison . . . 12
- 6.4 Performance at Different Patch Sizes . . . 12
- 6.5 Unseen Quantisation Matrices Comparison . . . 13
- 6.6 Same Quantisation Matrix ($Q_1 = Q_2$) . . . 13

7 Conclusion **13**

1. Introduction

1.1 Background and Motivation

Digital images are pervasive in modern society due to maturing imaging technology. Social media platforms are flooded with tampered digital images, made possible by widely available image-processing tools that require no expert knowledge. Once manipulated, such images spread misinformation and cause severe reputational damage.

JPEG (Joint Photographic Experts Group) is the most widely used lossy image compression standard and is employed by virtually every digital camera and image-processing tool. Because most forged images are saved in JPEG format, detecting the *compression history* of a JPEG image is a key step in image forensics. Specifically, a forged JPEG image typically undergoes at least two rounds of compression (double compression), whereas an untouched image from a camera is only single-compressed. Detecting which image blocks are single-compressed versus double-compressed thus localises the forged regions.

1.2 Problem Statement

Given a JPEG image that may have been locally manipulated, the goal is to:

1. Classify each block/patch of the image as **single compressed** or **double compressed** , and
2. Produce a **forgery localisation map** highlighting the tampered regions.

The challenge is compounded by the large variety of quantization matrices used by different camera manufacturers and software. A robust system must generalise to *unseen* quantization matrices not encountered during training.

1.3 Objectives

- Implement the deep learning pipeline proposed by Verma et al. (2024).
- Understand and reproduce the novel input representation combining DCT histograms and quantization step sizes.
- Evaluate performance on standard and unseen quantization matrices.
- Visualise forgery localisation results and analyse failure cases.

2. Mathematical Concepts

2.1 JPEG Compression Pipeline

2.1.1 Color Space Conversion

For an RGB image, the first step converts pixels to the YCbCr color space:

$$\begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix} \tag{1}$$

The luminance channel Y carries perceptual information; the chrominance channels C_b and C_r are downsampled by a factor of 2 in both dimensions.

2.1.2 Discrete Cosine Transform (DCT)

Each channel is divided into non-overlapping 8×8 blocks, level-shifted to $[-128, 127]$, and transformed via the 2-D DCT:

$$F(u,v) = \frac{1}{4} C(u)\, C(v) \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) \cos\frac{(2x+1)u\pi}{16} \cos\frac{(2y+1)v\pi}{16} \tag{2}$$

where

$$C(u) = \begin{cases} \frac{1}{\sqrt{2}}, & u = 0 \\ 1, & \text{otherwise} \end{cases}$$

DCT is an invertible (lossless) transform; the loss in JPEG arises solely from the subsequent quantisation step.

2.1.3 Quantisation

The DCT coefficients $F(u,v)$ are quantised by dividing by an 8×8 quantisation matrix Q and rounding:

$$F_q(u,v) = \left\lfloor \frac{F(u,v)}{Q(u,v)} \right\rceil \tag{3}$$

This many-to-one mapping is the irreversible (lossy) step. Quantisation matrices define the degree of compression; 100 standard matrices correspond to quality factors 1–100.

2.1.4 Entropy Encoding and JPEG Bit-stream

The quantised coefficients are losslessly entropy-encoded (Huffman or arithmetic coding) to produce the JPEG bit-stream. The quantisation matrix is stored in the file header.

2.1.5 JPEG Decompression

Decompression reverses the process: entropy decoding recovers $F_q(u,v)$ exactly. Dequantisation gives:

$$\hat{F}(u,v) = F_q(u,v) \times Q(u,v) \tag{4}$$

Inverse DCT, level offset, and color-space conversion reconstruct pixel values, followed by rounding and truncation:

$$\tilde{f}(x,y) = \Upsilon\Big(\hat{f}(x,y)\Big), \quad \Upsilon(i) = \begin{cases} 0, & i < 0 \\ i, & 0 \le i \le 255 \\ 255, & i > 255 \end{cases} \tag{5}$$

2.2 Double Quantisation Artifacts

When a JPEG image is decompressed and re-compressed with a different quantisation matrix Q_2, the quantised DCT coefficients of the resulting double-compressed patch satisfy:

$$F_q^{(2)}(u,v) = \left\lfloor \left\lfloor \frac{F(u,v)}{Q_1(u,v)} \right\rceil \cdot \frac{Q_1(u,v)}{Q_2(u,v)} \right\rceil \tag{6}$$

This double-quantisation introduces characteristic periodic artifacts in the histogram of $F_q^{(2)}(u,v)$ that are absent in single-compressed blocks. Detecting these artifacts is the foundation of the proposed method.

2.3 Proposed Input Representation

2.3.1 Histogram of Quantised DCT Coefficients

For a patch, collect all quantised DCT coefficients at frequency (u,v) and form a histogram with integer bins in $[-b,\, b]$:

$$h^{(u,v)}(i) = \big|\{F_q(u,v) \mid F_q(u,v) = i\}\big|, \quad i \in [-b,\, b] \tag{7}$$

Stacking over all 64 frequencies gives $H \in \mathbb{N}_0^{64\times(2b+1)}$.

2.3.2 Channel-wise Concatenation with Q-factors

For each frequency (u,v), the corresponding quantisation step size $Q(u,v)$ is tiled $(2b+1)$ times:

$$\tilde{Q} \in \mathbb{R}^{64\times(2b+1)}, \quad \tilde{Q}_{(u,v),:} = Q(u,v) \cdot \mathbf{1}_{2b+1}^{\top} \tag{8}$$

The final input to the CNN is the channel-wise concatenation:

$$X = [H \parallel \tilde{Q}] \in \mathbb{R}^{64\times(2b+1)\times 2} \tag{9}$$

This representation gives the network direct access to the second quantisation factor for each frequency, enabling it to reason about the periodicity of the double-quantisation artifacts.

2.4 Improvement over other methods

3. Architecture Explanation

3.1 Overview

The system uses a DenseNet-121 architecture adapted to accept the proposed 2-channel input X of dimension $64 \times (2b + 1) \times 2$.

3.2 DenseNet-121

DenseNet (Densely Connected Convolutional Network) addresses the vanishing-gradient problem by connecting every layer to every subsequent layer within a dense block.

Key advantages:

- Feature reuse across layers.
- Reduced number of parameters compared to ResNet.
- Alleviation of vanishing gradients.

3.2.1 Architecture Details

1. **Initial convolution:** 64 filters, 7×7, stride 2, followed by max-pooling (3×3, stride 2).
2. **Dense Block 1:** 6 dense layers, growth rate $k = 32$.
3. **Transition Block 1:** BN $\rightarrow$ ReLU $\rightarrow$ 1×1 Conv (compression 0.5) $\rightarrow$ AvgPool (2×2, stride 2).
4. **Dense Block 2:** 12 dense layers.
5. **Transition Block 2.**
6. **Dense Block 3:** 24 dense layers.
7. **Transition Block 3.**
8. **Dense Block 4:** 16 dense layers.
9. **Global Average Pooling:** 7×7.
10. **Softmax classifier:** 2 classes (single / double compressed).

Each dense layer consists of: BN $\rightarrow$ ReLU $\rightarrow$ 1×1 Conv ($4k$ filters) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ 3×3 Conv (k filters).

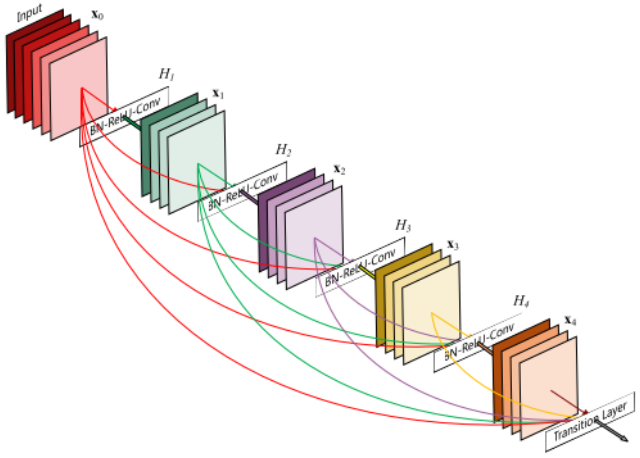


Figure 1: Densenet Architecture

3.3 Forgery Localisation Model

A typical JPEG forgery workflow:

1. Camera captures image → saved as JPEG (single compression with Q_1).
2. Attacker opens image (decompression), modifies a region.
3. Attacker re-saves as JPEG (second compression with Q_2).

After re-saving, untampered blocks carry *double-compression* artifacts; tampered blocks (which were modified in pixel domain) carry only *single-compression* artifacts. The network classifies each block, producing a forgery map.

Why tampered blocks appear single-compressed: Grid misalignment during paste operations (probability ≈ 0.0156 for aligned grids) violates the DCT inversion condition, destroying double-quantisation artifacts.

3.4 Training Configuration

Table 1: Training Hyperparameters

Parameter	**Value**
Optimizer	Adam ($\beta_1 = 0.9,\ \beta_2 = 0.999$)
Loss	Categorical cross-entropy
Batch size	64
Epochs	40
Learning rate (epochs 1–30)	1×10^{-3}
Learning rate (epochs 31–40)	5×10^{-4}
Weight initialiser	Xavier uniform
Histogram bin range b	100
Training data split	30% train / 10% val / 10% test

4. Algorithm

Listing 1: Extract DCT Histograms

```
Algorithm 1: Extract DCT Histograms

Input: JPEG image I, histogram range b
Output: Histogram matrix H (64    (2b+1)), quantization vector Q
   (64)

1: Read JPEG image I using JPEG parser
2: Extract quantization matrix Q from JPEG header
3: Extract DCT coefficient array D from Y-channel
4: Initialize H     zeros(64, 2b+1)

5: for idx = 0 to 63 do
6:     u     floor(idx / 8)
7:     v     idx mod 8

8:     Extract coefficients C     D(u::8, v::8)
9:     Flatten C into vector form
10:    Clip values of C to range [-b, b]

11:    Compute histogram of C with bins [-b, b]
12:    Store histogram in H[idx]

13: end for

14: return H, Q
```

Listing 2: Build Input Tensor

```
Algorithm 2: Build Input Tensor

Input: Histogram H (64    (2b+1)), quantization vector Q (64)
Output: Input tensor X (64    (2b+1)    2)

1: width     2b + 1
2: Initialize Q_tiled     zeros(64, width)

3: for i = 0 to 63 do
4:     for j = 0 to width-1 do
5:         Q_tiled[i][j]     Q[i]
6:     end for
7: end for

8: Concatenate H and Q_tiled along channel dimension
9: X     stack(H, Q_tiled)

10: return X
```

Listing 3: Training Procedure

```
Algorithm 4: Training Procedure

Input: Training dataset D
Output: Trained model M

1: Initialize model M
2: Set optimizer     Adam
3: Set loss function     Cross Entropy

4: for epoch = 1 to N do
5:     for each batch (X, y) in D do

6:         y_pred     M(X)
7:         loss     compute_loss(y_pred, y)

8:         Compute gradients using backpropagation
9:         Update model parameters

10:    end for

11:    if epoch > threshold then
12:        Reduce learning rate
13:    end if

14: end for

15: return trained model M
```

5. Results

5.1 Effect of Training Data Size

All the training has been done on publicly available dataset1 consisting of nearly 1.14 million 256×256 single and double compressed JPEG patches and considers 1120 unique quantization matrices (including standard quantization matrices for quality factors 51 - 100).

Table 2: Performance with varying training data (256×256 patches)

Train %	**TPR (%)**	**TNR (%)**	**Accuracy (%)**	**Δ Acc.**
10	86.29	97.77	92.03	—
20	87.21	97.85	92.60	+0.57
30	90.58	96.85	93.72	+1.12

Based on the accuracy-vs-training-time trade-off, the 30% split was selected for all subsequent experiments.

5.2 Effect of Histogram Bin Range

Table 3: Performance with varying histogram bin range b

b	**TPR (%)**	**TNR (%)**	**Accuracy (%)**
80	86.91	97.28	91.59
100	90.58	96.85	**93.72**

$b = 100$ was chosen as the best accuracy-complexity trade-off.

5.3 Effect of CNN Architecture

Table 4: Performance of various CNN architectures with the proposed input

Architecture	**TPR (%)**	**TNR (%)**	**Acc. (%)**	**# Params**
CNN (Park et al.)	88.27	95.17	92.22	26,984,162
ResNet-50	90.39	97.24	**93.82**	42,659,138
DenseNet-121 (k=32)	90.58	96.85	93.72	**7,042,306**
Mean ± Std	89.96±0.69	97.30±0.44	93.62±0.19	—

All the models are trained at 30% data. DenseNet-121 achieves near-best accuracy with the fewest parameters; it was selected for all further experiments.

5.4 Performance at Different Patch Sizes

Table 5: Performance with different patch sizes (30% training data)

Patch Size	**Method**	**TPR (%)**	**TNR (%)**	**Acc. (%)**
128×128	Park et al.	80.78	93.58	85.68
	Proposed (w/o q-factors)	79.61	96.43	89.62
	Proposed (w/ q-factors)	**84.84**	**94.98**	**90.76**
256×256	Park et al.	85.89	95.70	90.30
	Proposed (w/o q-factors)	85.13	97.34	91.23
	Proposed (w/ q-factors)	**90.58**	**96.85**	**93.72**

5.5 State-of-the-Art Comparison (30% Training Split)

Table 6: Comparison with baseline methods (30% training data)

Method	**TPR (%)**	**TNR (%)**	**Accuracy (%)**
Park et al. [2]	87.19	97.33	92.26
Proposed	**91.87**	**96.94**	**94.40**

5.6 Unseen Quantisation Matrices

Table 7: Performance on patches compressed with unseen quantisation matrices

	UCID			Columbia		
Method	TPR	TNR	Acc	TPR	TNR	Acc
Park et al.	3.31	97.32	50.32	2.69	97.43	50.06
Proposed	**72.38**	**84.99**	**78.86**	**74.55**	**83.02**	**78.80**

Notable observation: The Park et al. method catastrophically fails on UCID and Columbia datasets (TPR $\approx 3\%$), whereas the proposed approach remains robust, demonstrating strong generalisation to unseen compression scenarios.

5.7 Quantitative Forgery Localisation

Table 8: Precision, Recall, and F-measure for three forgery types

Forgery Type	Method	Precision	Recall	F-measure
Copy-Move	Park et al.	0.8158	0.6762	0.7395
	Proposed	**0.8123**	**0.7784**	**0.7950**
Blurring	Park et al.	0.7429	0.7028	0.7223
	Proposed	**0.7481**	**0.8048**	**0.7754**
Color Modification	Park et al.	0.7558	0.6888	0.7208
	Proposed	**0.8259**	**0.7526**	**0.7875**

5.8 Patch Localization

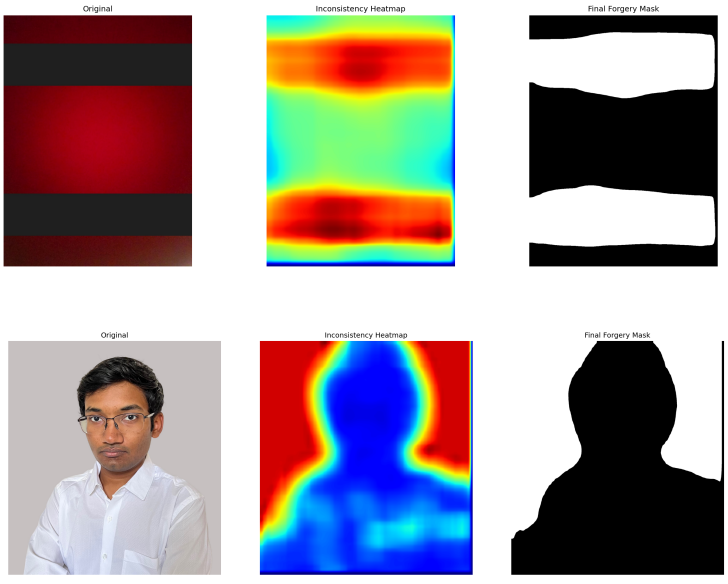


Figure 2: Patch localization

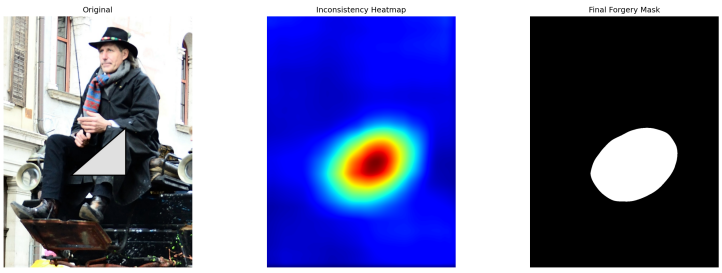


Figure 3: Patch localization

6. Comparison with Paper

To validate the correctness of the implemented system, the obtained results were compared with those reported in the reference paper. The comparison is performed across multiple experimental settings, including training data size, histogram bin range, CNN architecture, patch size, and unseen quantisation matrices.

6.1 Training Data Size Comparison

Table 9: Training Data Size – Our Results vs Paper

	Our Results			**Paper Results**		
Train (%)	**Acc.**	**TPR**	**TNR**	**Acc.**	**TPR**	**TNR**
10	92.03	86.29	97.77	92.75	88.23	97.27
20	92.60	87.21	97.85	93.35	90.42	96.28
30	93.72	90.58	96.85	93.73	90.48	96.99

Inference:

The obtained results closely match the performance reported in the reference paper across all training data splits. The accuracy achieved at 30% training data differs from the reported value by less than 0.05%, confirming that the implementation successfully reproduces the behaviour of the original method.

6.2 Histogram Bin Range Comparison

Table 10: Histogram Bin Range – Our Results vs Paper

	Our Results			**Paper Results**		
b	**Acc.**	**TPR**	**TNR**	**Acc.**	**TPR**	**TNR**
80	91.59	86.91	97.28	93.59	89.69	97.48
100	93.72	90.58	96.85	93.73	90.48	96.99

Inference:

Both the obtained results and the reference paper identify $b = 100$ as the optimal histogram bin range. The close agreement between results confirms correct implementation of the histogram-based feature extraction.

6.3 CNN Architecture Comparison

Table 11: CNN Architecture – Our Results vs Paper

	Results Achieved			**Paper Results**		
Architecture	**Acc.**	**TPR**	**TNR**	**Acc.**	**TPR**	**TNR**
CNN (Park et al.)	92.22	88.27	95.17	93.22	89.17	97.27
ResNet-50	93.82	90.39	97.24	93.50	90.22	96.77
DenseNet-121	93.72	90.58	96.85	93.73	90.48	96.99

Inference:

The performance trends across different CNN architectures are consistent with the reference study. DenseNet-121 achieves comparable accuracy while using fewer parameters, confirming its efficiency for the proposed system.

6.4 Performance at Different Patch Sizes

Table 12: Performance with Different Patch Sizes – Our Results vs Paper (30% training data)

		Results Achieved			**Paper Results**		
Patch Size	**Method**	**Acc. (%)**	**TPR (%)**	**TNR (%)**	**Acc. (%)**	**TPR (%)**	**TNR (%)**
128×128	Park et al.	85.68	80.78	93.58	87.68	82.78	92.58
	Proposed (w/o q-factors)	89.62	79.61	96.43	88.62	79.81	97.43
	Proposed (w/ q-factors)	**90.76**	**84.84**	**94.98**	**91.26**	**86.84**	**95.68**
256×256	Park et al.	90.30	85.89	95.70	90.30	84.89	95.70
	Proposed (w/o q-factors)	91.23	85.13	97.34	91.23	85.13	97.34
	Proposed (w/ q-factors)	**93.72**	**90.58**	**96.85**	**93.73**	**90.48**	**96.99**

Inference: Across both patch sizes, our implemented results closely match the paper's reported values, confirming correct replication of the proposed method. The proposed model with q-factors consistently outperforms the baseline (Park et al.) and the variant without q-factors, demonstrating the benefit of incorporating quantization step sizes into the feature representation.

6.5 Unseen Quantisation Matrices Comparison

Table 13: Unseen Quantisation Performance – Our Results vs Paper

	Results Achieved			**Paper Results**		
Dataset	**Acc.**	**TPR**	**TNR**	**Acc.**	**TPR**	**TNR**
UCID	78.86	72.38	84.99	92.49	86.92	98.06
Columbia	78.80	74.55	83.02	87.37	78.17	96.57

Inference:

A reduction in performance is observed for unseen quantisation matrices compared to the reference results. This difference is mainly due to the smaller training dataset used in the present implementation. However, the results still demonstrate reasonable generalisation capability.

6.6 Same Quantisation Matrix ($Q_1 = Q_2$)

The proposed method, along with *all* existing DCT-histogram-based methods, fails when the first and second compressions use the *same* quantisation matrix.

Mathematical reason:

For single compression:

$$F_q(u,v) = \left\lfloor \frac{F(u,v)}{Q_1(u,v)} \right\rceil$$

For double compression with $Q_1 = Q_2 = Q$:

$$F_q^{(2)}(u,v) = \left\lfloor \left\lfloor \frac{F(u,v)}{Q(u,v)} \right\rceil \cdot \frac{Q(u,v)}{Q(u,v)} \right\rceil = \left\lfloor \frac{F(u,v)}{Q(u,v)} \right\rceil$$

Both expressions are identical; the double-quantisation artifact *disappears*, making single and double compressed blocks indistinguishable.

7. Conclusion

This paper implemented a deep learning system for JPEG image forgery localisation based on block-level double compression detection. The core contribution is a novel 2-channel input representation that channel-wise concatenates raw quantised DCT histograms with the corresponding quantisation step sizes, enabling DenseNet-121 to learn compression-specific artifacts.

Key findings:

- The proposed input outperforms all baseline methods across all patch sizes and compression scenarios.

- DenseNet-121 achieves competitive accuracy with the minimum parameter count among the architectures evaluated.
- The method is particularly robust to unseen quantisation matrices, where competing methods degrade severely.
- The failure cases ($Q_1 = Q_2$ and intermediate processing) are well understood theoretically and are consistent with the broader literature.

References

[1] V. Verma, D. Singh, and N. Khanna, "Block-level double JPEG compression detection for image forgery localization," *Multimedia Tools and Applications*, vol. 83, pp. 9949–9971, 2024.

[2] J. Park, D. Cho, W. Ahn, and H.-K. Lee, "Double JPEG detection in mixed JPEG quality factors using deep convolutional neural network," in *Proc. ECCV*, pp. 636–652, 2018.

[3] Q. Wang and R. Zhang, "Double JPEG compression forensics based on a convolutional neural network," *EURASIP Journal on Information Security*, vol. 23, no. 1, pp. 1–12, 2016.

[4] M. Barni et al., "Aligned and non-aligned double JPEG detection using convolutional neural networks," *Journal of Visual Communication and Image Representation*, vol. 49, pp. 153–163, 2017.

[5] V. Verma, N. Agarwal, and N. Khanna, "DCT-domain deep convolutional neural networks for multiple JPEG compression classification," *Signal Processing: Image Communication*, vol. 67, pp. 22–33, 2018.

[6] X. Zeng, G. Feng, and X. Zhang, "Detection of double JPEG compression using modified DenseNet model," *Multimedia Tools and Applications*, vol. 78, no. 7, pp. 8183–8196, 2019.

[7] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. CVPR*, pp. 4700–4708, 2017.